Gmail Vamshi Pokala <vam.pokala@gmail.com>

How To Reduce Inference Costs While Running LLMs

1 message

Sahib Dhanjal from To Data & Beyond <youssefh@substack.com> Wed, Feb 4, 2026 at 9:27 PM
Reply-To: Sahib Dhanjal from To Data & Beyond
<reply+337xd9&5isyun&&94e7d02cdde1e7067596ef7d35bb0f315458c4cc7e8fbc970f0038e44b2cc7de@mg1.substack.com>
To: vam.pokala@gmail.com

How To Reduce Inference Costs While Running LLMs

The only guide you need to inference scaling for AI models

SAHIB DHANJAL
FEB 5 • GUEST POST

READ IN APP

Get 30% off forever

Inference scaling refers to the ability to run model predictions efficiently and reliably as usage grows. Unlike training — which happens once or intermittently — inference happens every time a user makes a query or a system generates an output. In production applications (like Gemini, Claude, etc), that number ranges anywhere from the thousands to millions of operations per day. Without employing smart scaling techniques, costs would easily spiral out of control, and your $20/month Gemini Pro subscription would be more like $200/month.

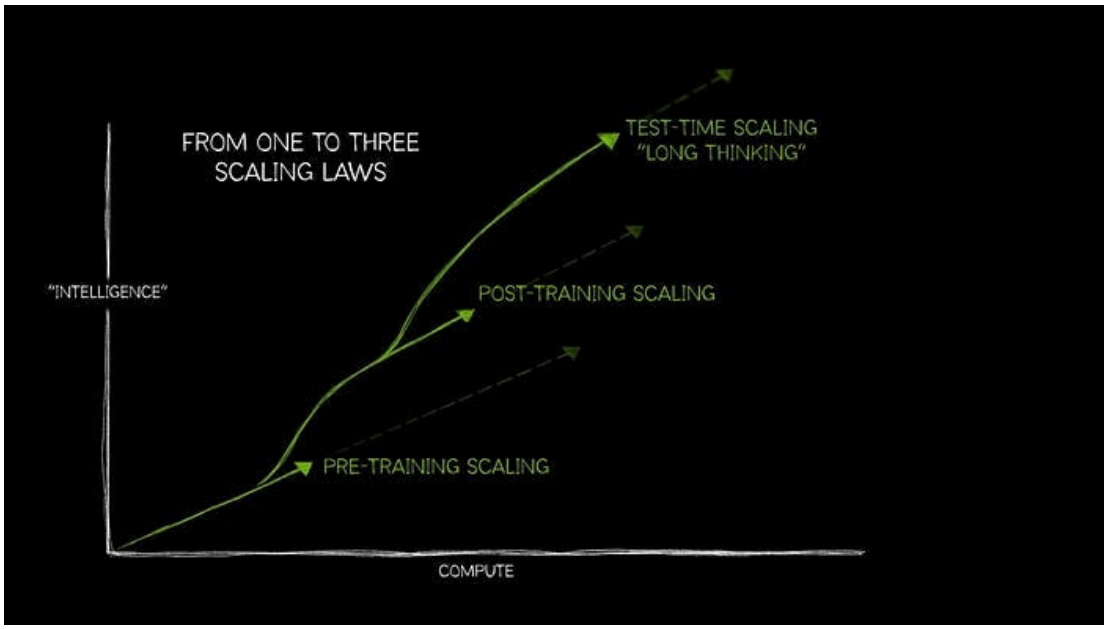
This increase in cost is attributed to the following couple of reasons:

1. **GPU Hours**: GPUs are the single largest cost driver, whether you're self-hosting or using cloud vendors. Larger models require much more

VRAM and compute, which in turn pushes you to buy or rent expensive hardware.

2. **Token Processing**: Every LLM generates output token by token. In other words, this basically means that each new text generation requires recomputing context, which adds up when outputs are long, and user traffic is high.
3. **Memory Bandwidth and Data Movement**: Models are memory-bound: moving data in and out of VRAM costs time and money — especially in distributed systems. Reducing this movement requires chip manufacturers and hyper-scalers to build custom hardware to reduce this movement as much as possible.
4. **Idle GPU Time**: Without batching or dynamic scaling, GPUs sit idle waiting for requests — wasting money. Efficient utilization is literally the name of the game during inference.

As unfortunate as it might be, models don't scale linearly in cost — they scale with compute time, token count, hardware choice, utilization efficiency, and system topology. Understanding and optimizing these components is what keeps your subscription limited to $20/month and not more.



To Data & Beyond is a reader-supported publication.
To receive new posts and support my work, consider becoming a free or paid subscriber.

✔Subscribed

Table of Contents:

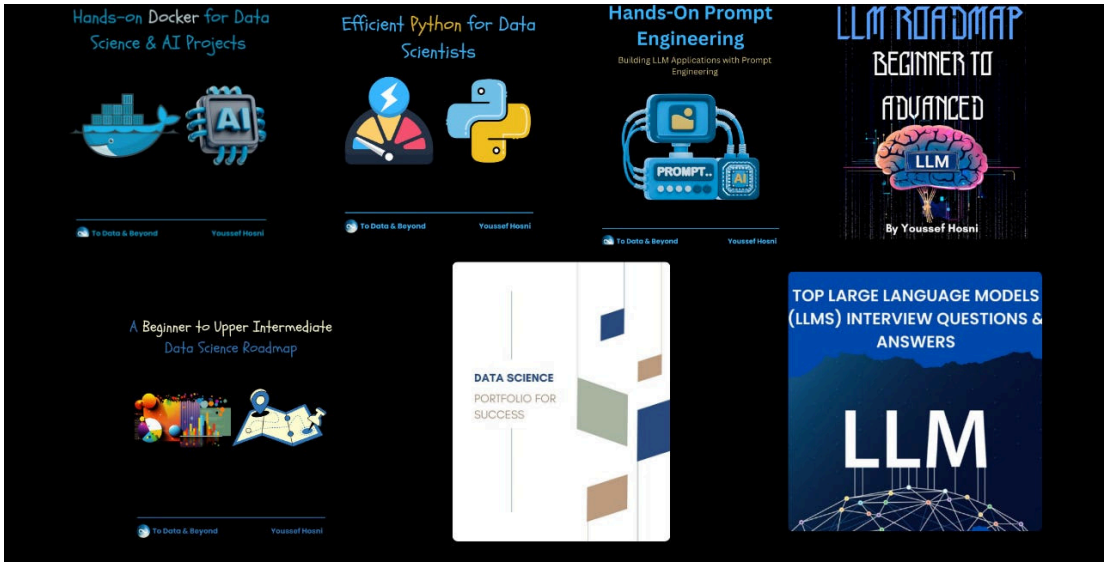
c. Design-Based Scaling — The “System of Systems”
d. Model-Based Scaling
e. Prefill-Decode Separation
f. Multi-LoRA Serving
g. Flash Attention

12. Conclusion

Get All My 8 Books With 60% Off

Get All My 8 Books, One Button Away With 60% Off

YOUSSEF HOSNI · JUNE 17, 2025



I have created a bundle for my books and roadmaps, so you can buy everything with just one button and for 40% less than the original price. The bundle features 8 eBooks, including:

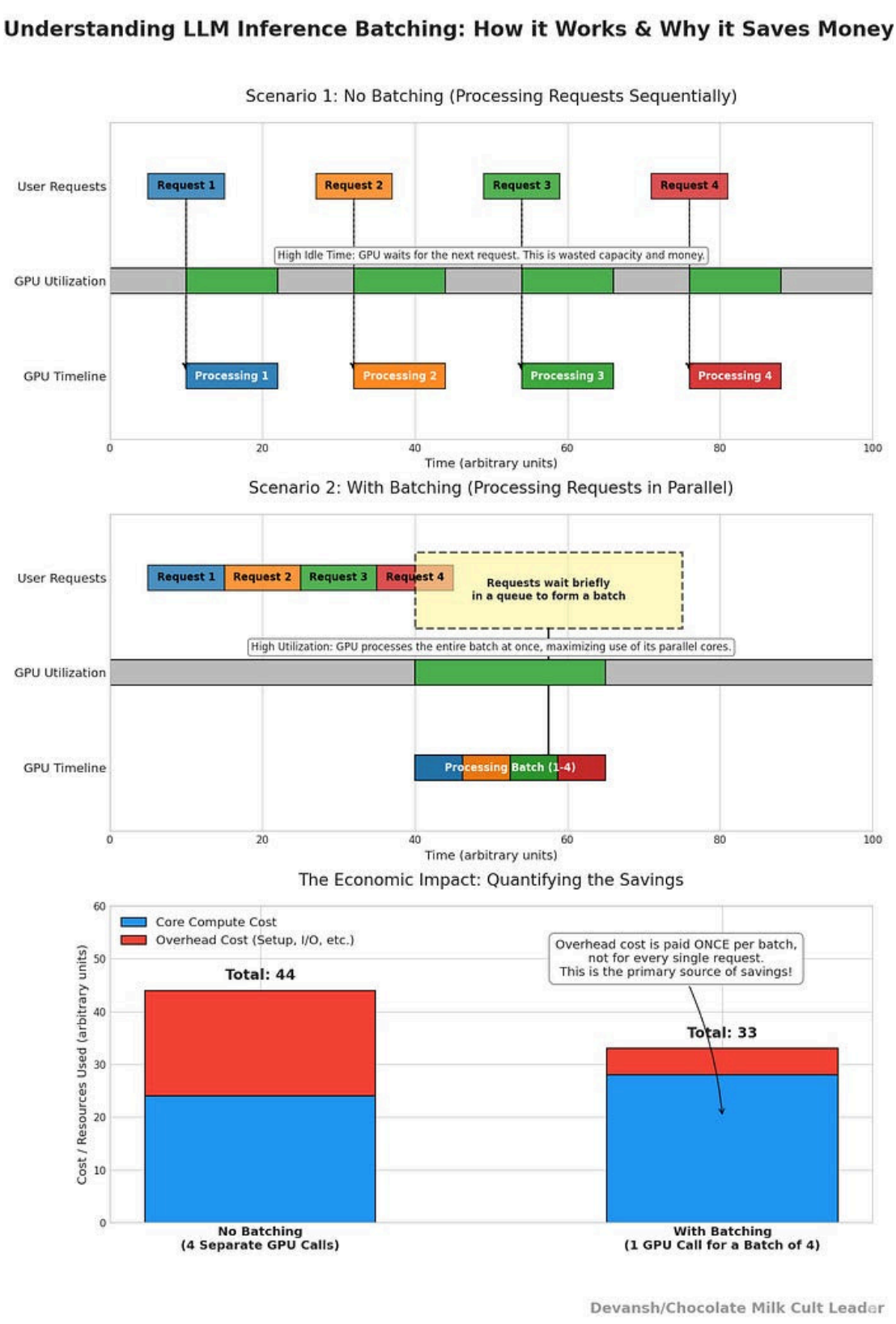
Read full story

---

1. Batching and Parallelism — The Primary Throughput Driver

At the hardware level, LLMs are memory-bound, not compute-bound. When you run a single request, the GPU spends most of its time waiting for data to move from VRAM to the processors, which, in all honesty, is nothing but wasteful. To tackle this, we can employ the following approaches:

1.1. Continuous Batching



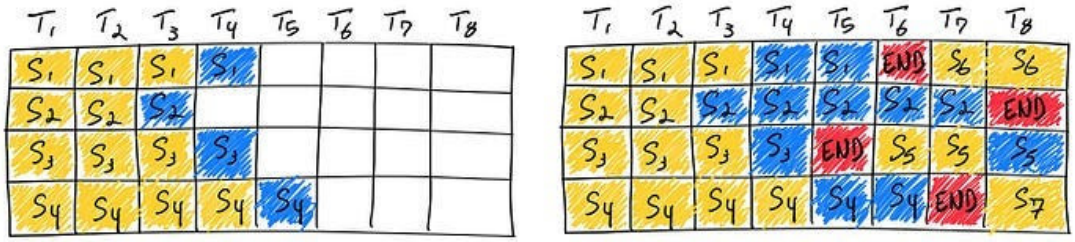
Understanding batching for LLM inference [Source]

Instead of waiting for a whole batch to finish, continuous batching inserts new requests the moment a token is generated for an existing one. This significantly improves computational efficiency and resource utilization

compared to processing requests one-by-one. When you make a call to your model, 2 things happen under the hood:

1. **Prefill** — Process the entire input prompt at once. This is matrix multiplication-heavy, hence more sequences in parallel imply higher GPU utilization.
2. **Decode** — Generates tokens one at a time. Each token depends on the history of prior tokens, which is dominated by memory read/writes.

At every token generation step, the system checks if any sequence in the batch has completed. If any of them have completed, new requests from the queue are inserted into the GPU asap, ensuring the GPU remains saturated with active computations.



With iteration-level scheduling, the next request is scheduled as soon as a request finishes [Source]

This is the staple when it comes to techniques that provide magnitudes in terms of returns with minimal effort. There have been multiple improvements to batching techniques (for example, multi-bin batching) in order to squeeze even more performance out of GPUs, but that deserves a whole chapter in itself. In case you're interested in learning more about how continuous batching works, I'll defer you to this detailed article by Hugging Face.

1.2. Multi-GPU Parallelism

Pipeline Parallelism Illustration [Source]

Most state-of-the-art models are so huge that they don't fit onto a single GPU. This is where multi-GPU parallelism comes into the picture! Techniques like **Tensor** and **Pipeline Parallelism** are often employed during inference, which split the model across multiple GPUs, presenting the following benefits — ability to handle much larger models, and increase throughput by parallelizing the heavy matrix multiplications. If you're not aware of what either of these is or how they work, you should definitely check out this article from my archive.

---

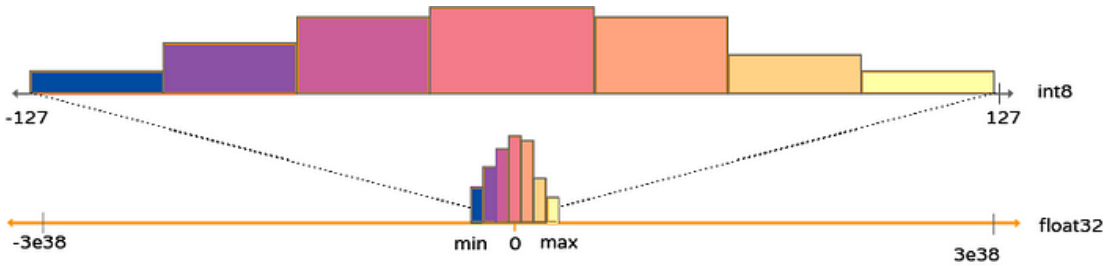
2. Deep Learning Compilers and Graph Execution

Python's flexibility is a bottleneck in production. Under the hood, libraries like PyTorch/TensorFlow issue thousands of GPU kernel calls with intermediate results being written to the GPU's global high bandwidth memory (HBM) and subsequently the registers. All these reads and writes add a lot of unnecessary overhead during inference. This is where inference compilers (Nvidia TensorRT, ONNX Runtime, etc) and kernel fusion come in.

By merging multiple operations into a single kernel, we effectively eliminate the overhead of data movement between the GPU's registers and memory, as well as reduce the number of kernel launches. If you want to dive a little deeper into how compilers work, or what they do, I have an article on that, too.

---

3. Quantization — Doing More with Less



Representative mapping from FP32 to INT8 [Source]

More often than not, you do not need the entire 16/32-bit precision of the model weights to maintain accuracy. Quantization reduces the precision of model weights (e.g., from FP32/16 to INT8 or even 4-bit). Hybrid quantization approaches, with proper calibration, have shown negligible performance loss while reducing the memory footprint of your model by 4x, allowing you to fit your 70B parameter model onto a single GPU instead of a GPU cluster. New standards like Nvidia's FP4 (on Blackwell chips) even allow for 50% less memory usage than FP8 with virtually zero accuracy loss, effectively doubling the context length a single GPU can handle. The math is simple:

> smaller weights = less memory traffic = more tokens/sec

Once you're able to run your quantized model with <1% regression on your evaluation dataset, consider your job done.

4. KV Cache Optimization

Transformers store two vectors for each token at every layer: the key (K) and value (V). Together, these form the KV cache without which every token would have to be recomputed from scratch. This, however, grows linearly with context length, representing the classic software engineering tradeoff of using memory to save on compute. This is where techniques such as Paged Attention, Context Caching, Grouped Attention, and the like come in. Without going into too much detail, the following techniques are considered the industry standard:

4.1. Paged Attention

Borrowing from operating system virtual memory, Paged Attention allows the KV cache to be stored in non-contiguous memory blocks (similar to virtual memory). This eliminates fragmentation and allows for much higher concurrency, making it much easier to reuse common prefixes.

4.2. Context Caching

More often than not, the system prompt (specifically in chat-based applications) is the same across multiple prompts. Context Caching (also known as Prompt or Prefix Caching) works by identifying and caching frequently reused text (**Prefill** phase)— like long system instructions, multi-turn chat histories, or RAG documents — and pointing subsequent requests at the same data.

If you send a request that starts with the same 2,000-token system prompt you used a minute ago, the server doesn’t re-read those 2,000 tokens. It looks up a hash of the text in its memory, finds the pre-computed KV tensors, and jumps straight to the **Decode** phase. Some of the benefits it provides are:

- **Time-to-First-Token (TTFT)Reduction:** For long prompts, the “prefill” phase can take several seconds. Context caching can drop this to milliseconds.
- **Massive Cost Savings:** Since the server does almost no work for cached tokens, the price can be up to 90% cheaper than standard input tokens. Major providers like Anthropic and OpenAI now offer discounted pricing for these cached tokens.
- **Persistent Multi-Turn Context:** In chat-based applications, every time you send a message, the entire previous history is sent again. Without caching, you pay for the *entire* history every single turn. With caching, you only pay for the *new* message and the generation.

4.3. Multi-Query (MQA) and Grouped-Query Attention (GQA)

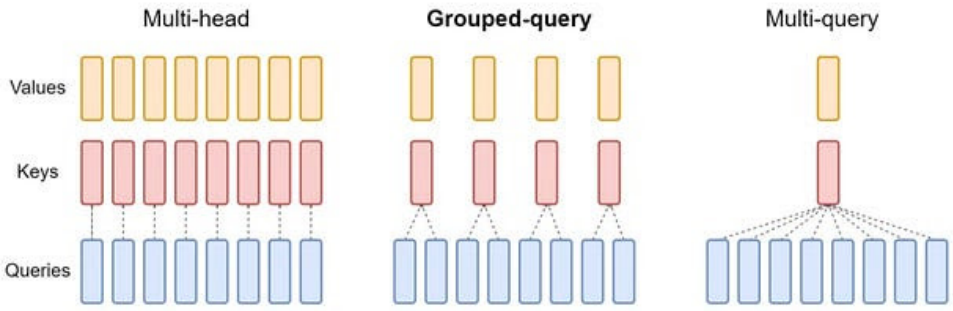


Figure 2: Overview of grouped-query method. Multi-head attention has H query, key, and value heads. Multi-query attention shares single key and value heads across all query heads. Grouped-query attention instead shares single key and value heads for each *group* of query heads, interpolating between multi-head and multi-query attention.

Grouped Query and Multi-Query Attention Heads [Source]

Instead of each query head having its own KV head, all query heads share a single KV head. With 32 or 64 heads, that means 32 or 64 copies of the same information. With MQA, if your model has 32 attention heads, you still have 32 query heads, but only 1 key head and 1 value head. On the contrary, instead of sharing 1 KV head across all queries, GQA divides the query heads into groups. Each group shares a single pair of KV heads. This can reduce cache size by 8× or more if done properly.

4.4. KV Quantization

Similar to model weight quantization, we can store K/V in INT8 or INT4 instead of FP16, cutting the storage and bandwidth by half or more. While model quantization is well-understood, KV quantization is trickier. If you compress too hard, the model starts losing its train of thought and sometimes begins repeating itself.

That said, modern techniques like KIVI and FlexGen use “outlier-aware” quantization — they keep a tiny fraction of the most important tokens in high precision while crushing the rest. This allows for massive memory savings with less than a 1% drop in model accuracy.

4.5. Forgetful Attention — The Local Focus Strategy

Instead of forcing every token to look at every other token in a massive sequence, the Sliding Window Attention technique restricts each token’s attention to a fixed-size **window (W)** of its immediate neighbors (for example, the last 4096 tokens).

While this is more of a dumb fix (as it forgets everything once it falls out of the window), **Forgetful Attention** is smart. It uses a **Forget Gate** — a mechanism borrowed from older RNN architectures — to decide which tokens are actually useless.

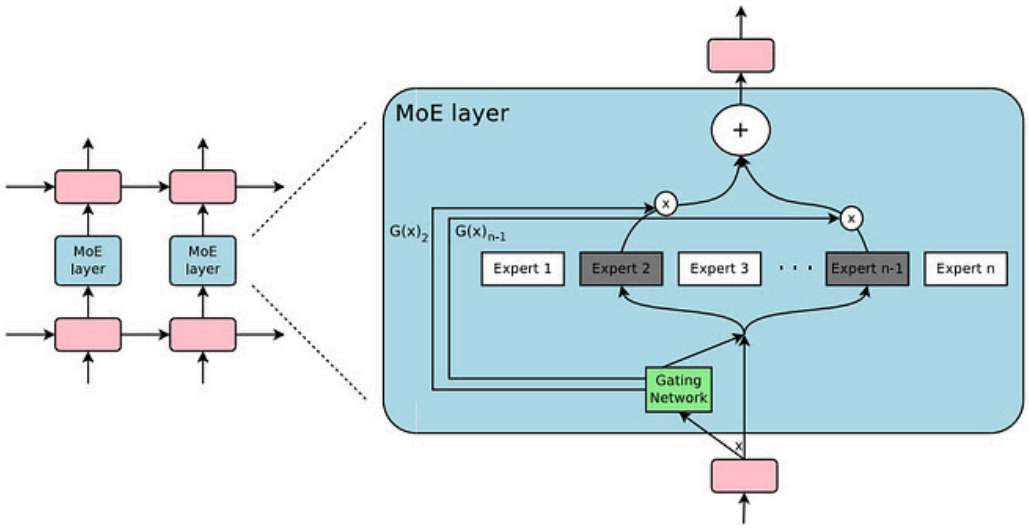
---

5. Towards Sparse Architectures

Transformer-based architectures are extremely dense. Every parameter in the model fires on every token, scaling FLOPS almost linearly with model size and almost quadratically with context length. This is fine for smaller

models, but it isn't really usable in production. This is where sparsity comes in, letting you add capacity or length without paying dense costs. There are two approaches to tackle this:

5.1. Mixture-of-Experts Models



Mixture of Experts (MoE) Layer [Source]

Why run 1.8 trillion parameters if you only need 17 billion for a specific prompt? **Mixture of Experts (MoE)** models use "routers" to activate only a small subset of the model per token. Essentially, they are networks composed of expert sub-networks and a gating network that dynamically routes inputs to the appropriate experts. This allows for conditional computation, making large networks much more efficient.

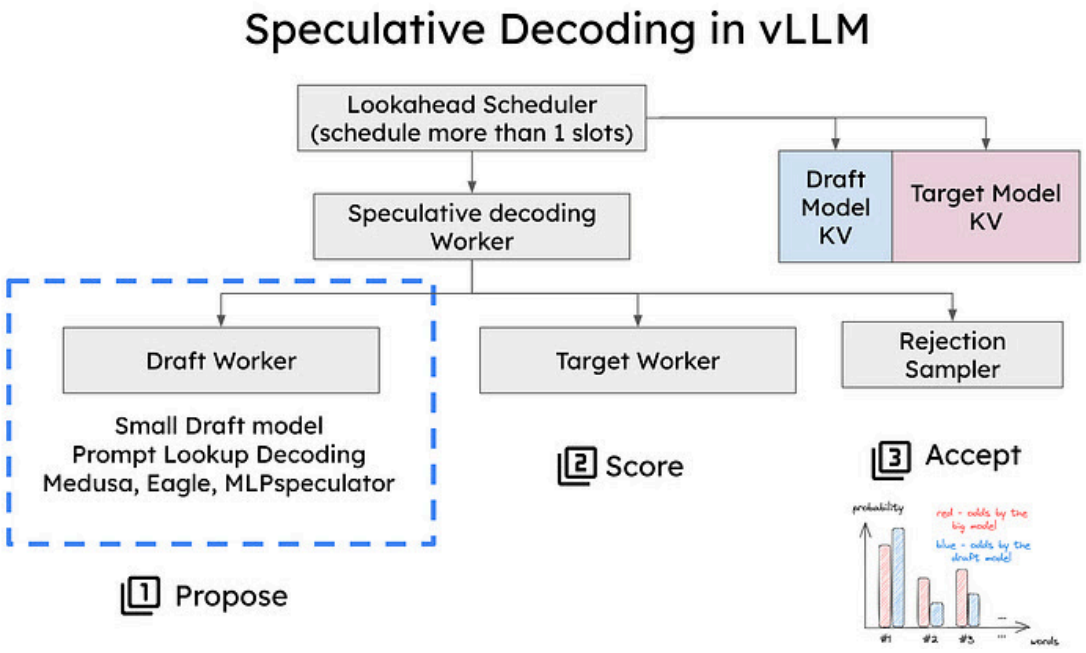
The LLaMA-4 family of models popularized this approach by using specialized "mini-models," or experts (scaling up to 128 of them). This allows the model to retain massive knowledge while activating only a small subset at inference time, leading to roughly a 3× reduction in FLOPs per token compared to dense models. Hugging Face has a very detailed article about this if you're interested in reading more about it.

5.2. Sparse Attention

Sparse Attention follows the "less is more" philosophy applied to the attention matrix. Instead of every token attending to every other token, the model only

computes interactions for a small, strategic subset. This reduces the complexity from $O(N^2)$ to near-linear $O(N\log N)$ or even $O(N)$.

6. Speculative Decoding — The Draft and Review Strategy



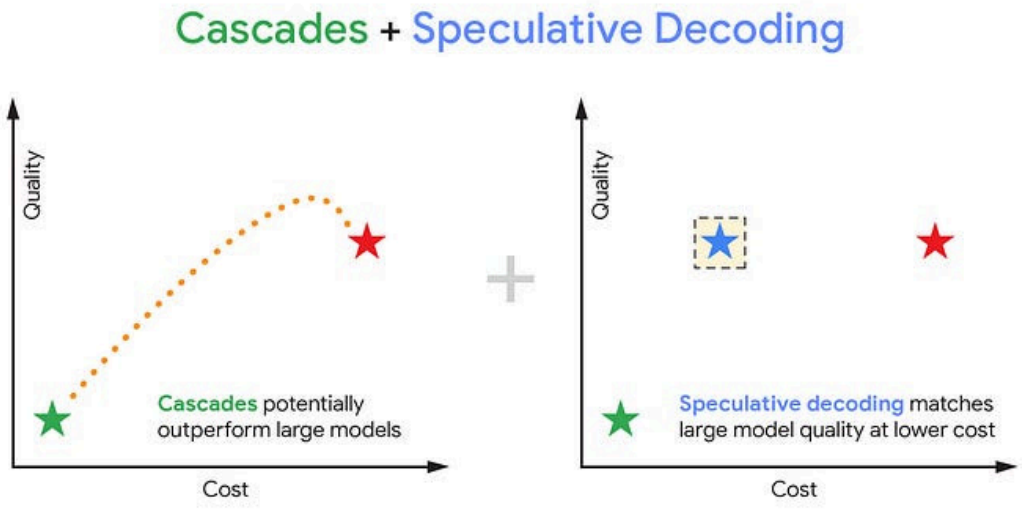
System Architecture of Speculative Decoding [Source]

Speculative decoding is an optimization technique that speeds up inference by using a small, cheap **assistant** model to guess tokens before a larger, expensive **expert** model verifies them. It works as follows:

1. **Drafting (Small Model)** — A tiny, fast model quickly "guesses" multiple future tokens (let's say n tokens). Because it's small, it can do this very cheaply and rapidly.

2. **Verification (Large Model)** — The large model performs a single forward pass over the original context + the n tokens generated by the drafting model. If this model agrees with the first couple of guesses (let's say k), it generates a next token of its own. In this way, you get k+1 tokens for the price of just 1 expensive memory-loading cycle.

There have been multiple ways to achieve this as well, sometimes achieving a performance boost of almost 2.8x in LLMs. This tool is often highly effective in both low/high-QPS environments, hence making it a versatile approach for increasing efficiency and reducing latency.



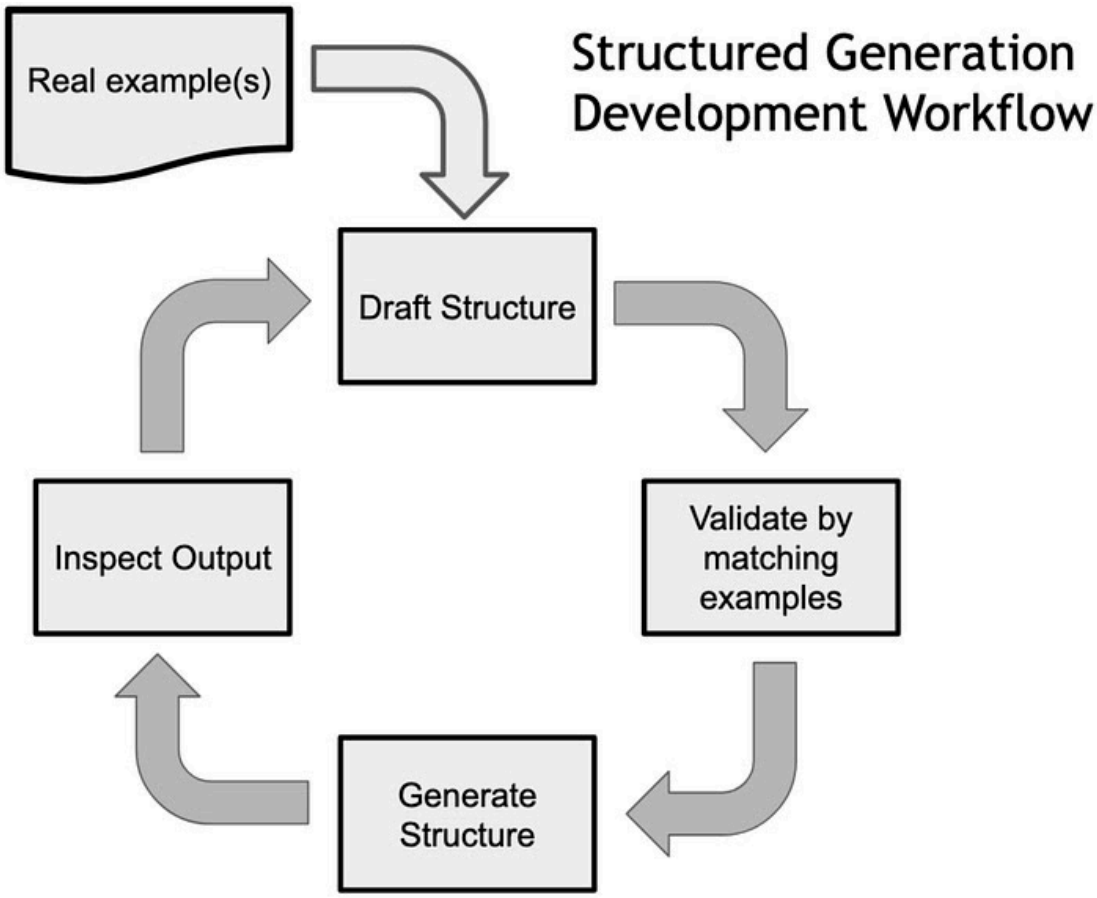
Speculative Cascades [Source]

Google research has its own flavor of this called speculative cascades, in which if the smaller model is confident, it finishes the task itself. If not, it drafts for the bigger model. This ensures you only pay for the "expensive" brain when necessary.

7. Structured Generation — Forcing Correctness

Structured Generation (using Constrained Decoding) is about making models controllable. In simple terms, structured generation is a technique that forces an LLM to follow a specific set of rules, such as a JSON schema, a regular expression (regex), or a programming language's grammar. When an LLM outputs JSON or code, it can hallucinate or waste tokens on conversational filler. While it might seem like adding a **checker** would add overhead, it actually reduces total system costs in three major ways:

- **Eliminates "Retry" Loops:** Without constraints, an LLM might generate a slightly broken JSON (maybe a missing comma). In a production app, this causes an error, forcing the system to **re-run the entire prompt**, which doubles your cost.
- **Enables Smaller Models:** You can often get "GPT-4 level" structured data out of a much smaller, cheaper model (like Llama-8B) by using constraints. The constraint acts as a **guardrail** that prevents the smaller model from wandering off-topic or hallucinating incorrect formats.
- **Reduced Output Length:** By forcing the model to strictly follow a schema, you prevent **chatter** — the model's tendency to explain itself. Fewer tokens generated equals lower cost.



Data Flow during Structured Generation [Source]

This is usually done using **Finite-State Machine** tools like **Guidance** or **Outlines** to "force" the model to pick only valid tokens. For a detailed overview of structured generation with constrained decoding, I'd highly encourage you to go through this blog by Aidan.

---

8. Teacher Student Distillation

This is a model compression technique where a large, high-performing **teacher** model is used to train a much smaller, more efficient **student** model. Instead of just learning from raw data, the student model is trained to mimic

the teacher's "soft" internal logic — specifically its probability distributions and reasoning patterns.

This allows the smaller model to capture the complex decision-making of the giant model without needing the same massive hardware to run. Since the smaller model has fewer parameters, it can easily fit into GPUs with lesser VRAM, providing lower latency, faster throughput,t and higher energy efficiency.

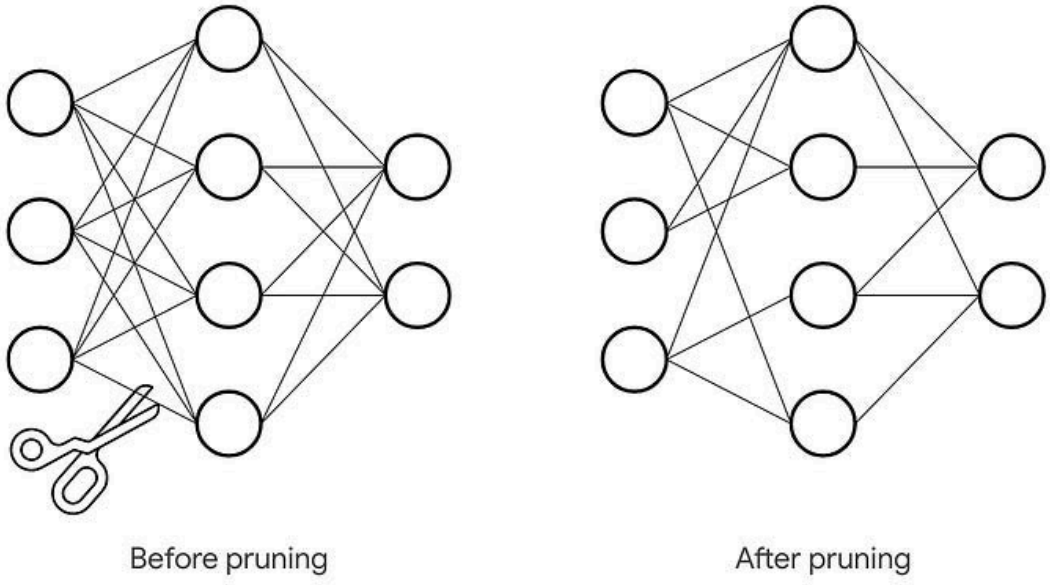
There are a few ways to do this:

- **Logit Distillation** — student matches the teacher's probability distributions. This is the most effective version of distillation, and ≥80% of your distillation budget should go here.
- **Response Distillation —** student mimics outputs directly.
- **Alignment Distillation** — students inherit guardrails from bigger aligned models.

One of the very best examples of this technique is Gemini 3 Flash, which is essentially a distillation of the much larger and much more powerful Gemini 3 Pro!

Get All My 8 Books With 60% Off

9. Model Pruning



Model Architecture after Unstructured Pruning [Source]

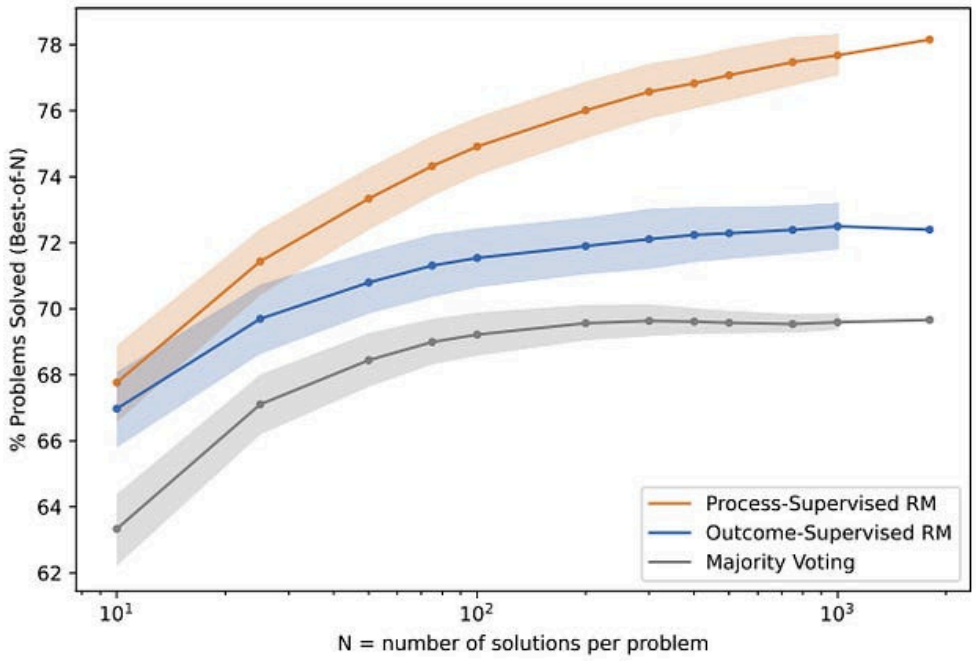
Model Pruning is a technique that reduces the size and computational complexity of an LLM by identifying and removing “redundant” or unimportant parts of the model’s internal structure. It is essentially a surgical way of “trimming” the neural network to create a more compact version that performs almost as well as the original. There are 2 types:

- **Unstructured Pruning:** Here, individual weights are set to zero, creating “sparsity,” but standard hardware often still processes those zeros, so actual speed gains can be limited without specialized kernels.
- **Structured Pruning:** Entire components like neurons, attention heads, or even full layers are removed physically. This makes the model significantly smaller, leading to immediate speed improvements on standard GPUs.

I have an entire article about different pruning techniques if you’re interested in learning more.

10. Process Reward Models (PRM)

	ORM	PRM	Majority Voting
% Solved (Best-of-1860)	72.4	**78.2**	69.6



Performance Metrics — ORM vs PRM [Source]

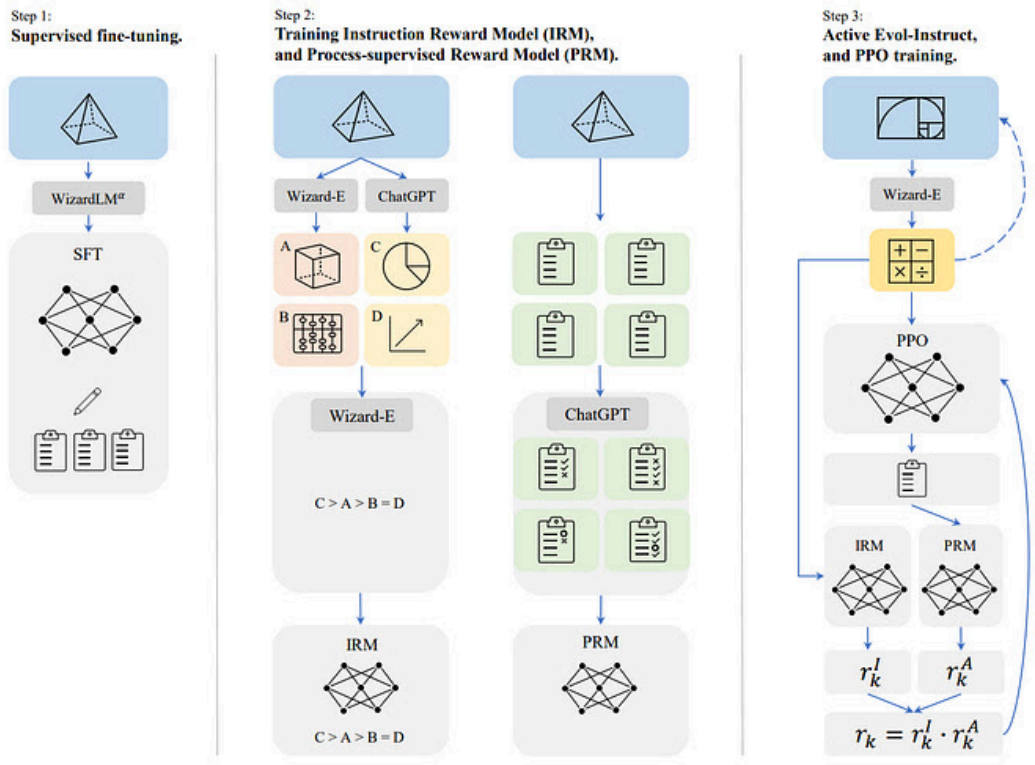
This is basically an algorithmic shortcut where, instead of checking the final answer using Outcome Reward Models (ORM), a verifier checks every step of the logic. If a path is wrong, it's pruned immediately.

This search + verify approach allows a smaller base model to reach the accuracy of a model 10x its size by thinking longer. PRMs are a core secret behind the massive performance jumps in reasoning models like OpenAI-O1 and DeepSeek-R1. PRMs not only offer faster convergence and training benefits but also help in inference-time scaling, aka (Test Time Compute), as a smaller model can "think" more:

- **Best-of-N Sampling —** A model generates 10 different reasoning paths. The PRM scores all 10 and picks the one with the highest step-by-step

score.

- **Search Guidance —** During "thinking," the PRM can act as a navigator. If a model starts a reasoning path that the PRM flags as "bad," the model can stop early and backtrack to a previous step, saving the cost of finishing a doomed response.
- **Smaller Model —** PRMs allow smaller, cheaper models to beat much larger ones. A 7B model using a PRM to verify its steps can often outperform a 70B model that is just guessing blindly.



Reinforcement Learning from Evol-Instruct Feedback (RLEIF) process [Source]

11. Marginal Improvements

Even the most optimized models live on distributed systems. These systems include CPUs, GPUs, Memory, Networks, and Caches amongst so many other components. While these aren't obvious bottlenecks (optimizing FLOPs, Memory, and Model Architecture), small optimizations like these can sometimes take you from 80 to 100 real-quick.

11.1. Topology Aware Inference

Where you place your model and how data flows through the different components often dictates how fast inference can be. **Topology-aware inference** optimizes how data flows between GPUs. By minimizing "hops" across the network (using NVLink or InfiniBand), you can reduce tail latency (slow individual responses) and ensure the cluster isn't stalled by network bottlenecks.

11.2. Policy Tuning

Policy Tuning via direct preference optimization **(DPO)** or reinforcement learning from human feedback **(RLHF**) can penalize verbosity. If a model provides an accurate answer in 50 tokens instead of 200, you've just cut costs by 75%. Organizations are now tuning models specifically to be "cost-aware" and concise.

11.3. Design-Based Scaling — The "System of Systems."

Design-based scaling uses a **Router Agent** to manage a fleet of models.

- **Tiered Inference:** Easy queries go to a smaller model; complex ones go to the larger model.
- **Context Caching:** As discussed in a previous section, reusing computed states for system prompts that never change. If your system prompt is 1,000 tokens and is used in every call, caching it can reduce your prefill costs by 90%+.

11.4. Model-Based Scaling

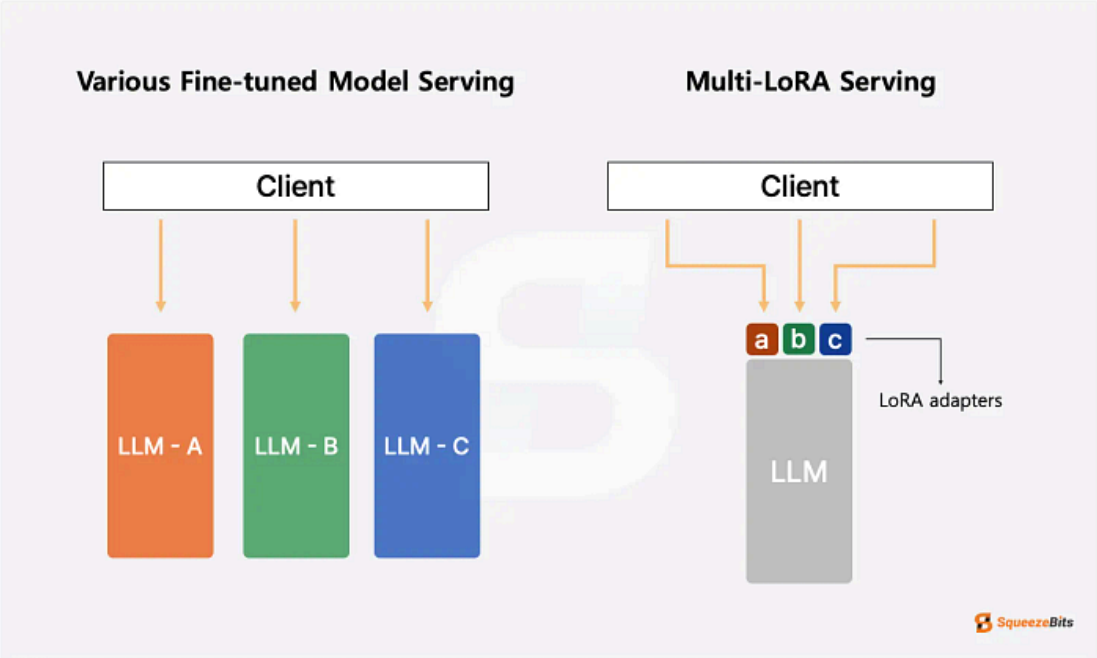
Ground-up architectural innovations are replacing the standard Transformer. For instance,

- **MatMul-Free Models:** New research shows LLMs can be built without matrix multiplication (using ternary weights like -1, 0, 1), leading to 10x memory efficiency and human-brain-level power consumption.
- **Receptance Weighted Key Value (RWKV) & Linear Attention:** Combines the fast training of Transformers with the constant-memory inference of RNNs.
- **Latent Reasoning:** Models like Coconut allow the AI to reason in "hidden math" (latent space) rather than writing out every word, which could make complex reasoning exponentially faster.

11.5. Prefill-Decode Separation

The prefill phase (ingesting your prompt) is compute-heavy, while the decode phase (typing the answer) is memory-bound. High-end labs now split these tasks onto separate GPU clusters. One cluster is optimized for high throughput (prefill node), which then streams the KV cache to a decode node optimized for low-latency token generation. This separation can increase overall hardware efficiency by over 30%.

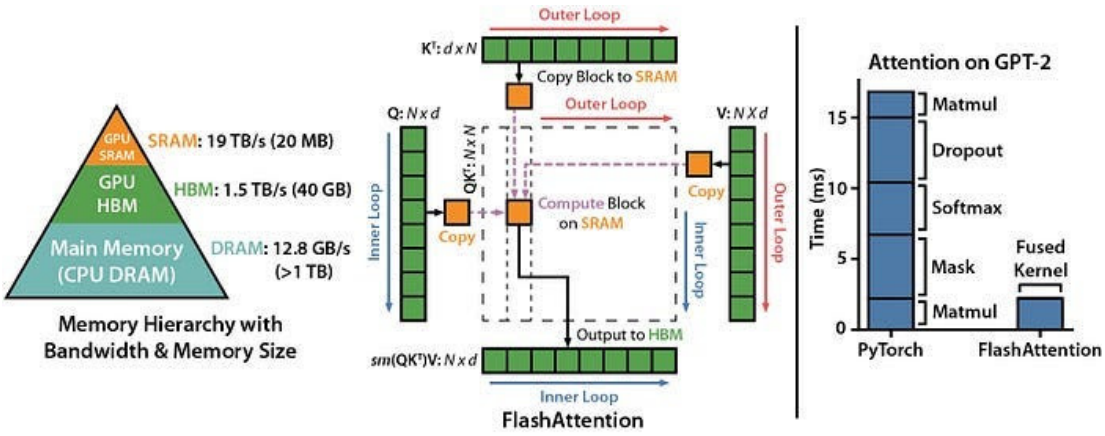
11.6. Multi-LoRA Serving



Multi-LoRA Serving [Source]

Instead of hosting 10 different fine-tuned models for 10 different tasks, companies host one **Base Model** and dynamically swap tiny **LoRA adapters** in and out of the GPU memory. This enables you to serve thousands of custom models using the memory footprint of just one, making **Personalized AI** economically viable.

11.7. Flash Attention



The Flash Attention Architecture [Source]

While most of the above optimizations focused on reducing the number of calculations, **Flash Attention** tackles a different problem altogether. As discussed previously, the GPU often spends more time moving data between its massive but slow HBM and its tiny but lightning-fast on-chip SRAM than it does actually performing MatMul operations. Flash Attention solves this constant back-and-forth data movement through a clever technique called **Tiling**.

Instead of trying to process the whole attention matrix at once, it breaks the Query, Key, and Value matrices into small blocks (**tiles**) that fit perfectly onto the GPU's SRAM, and performs all the necessary calculations locally within those blocks before writing only the final result back to memory, thereby reducing memory access by up to 10x. This "magic" trick is what allows modern LLMs to handle context windows of 100k+ tokens without crashing your hardware.

Conclusion

As we moved through 2025 (and into 2026 — Happy New Year!), The AI industry has undergone a **Great Efficiency Migration**. Research on brute-force training techniques is slowly, but intentionally, being replaced by an era of sophisticated, tiered, and highly optimized inference stacks, where the primary challenge is no longer just making a shiny new smart model, but also making that intelligence economical.

> Intelligence has become so cheap that it is now ubiquitous.

For enterprises, this shift marks the difference between a cash-draining pilot and a production-ready system that delivers increasing ROI over time, ultimately paving the way to profitability. For end users, these advances mean that larger, more capable models can be served at near-zero marginal cost — effectively commoditizing intelligence without the burden of recurring subscriptions.

In layman's terms, it's via these techniques that corporations like Google, Anthropic, and OpenAI are capable of offering access to their most advanced models essentially for free.

Enjoyed this post? Help me share this knowledge with others by clapping and sharing your thoughts. You can follow me on **Medium** / **LinkedIn** for more insights on C++, Python, Robotics and Trading algorithms.

Get All My 8 Books With 60% Off

A guest post by

Sahib Dhanjal

Invite your friends and earn rewards

If you enjoy To Data & Beyond, share it with your friends and earn rewards when they subscribe.

Invite Friends

LIKE COMMENT RESTACK

Get the app Start writing